

Naval Convoy Protection

Data-format pipeline: from raw drone telemetry to the L1 **ConvoyAdvance** command. Each stage shows the schema, the data type, and a real example value from the boot-bravo-safe proving run. Colour key: **off-chain**, **L2 (StarkNet/Madara)**, **L1 (Geth PoA)**.

Stage 0 Raw drone telemetry		OFF-CHAIN
Field	Type	Example / Notes
(x, y)	grid coords	Per-cell position from GPS / indoor UWB. e.g. (3, 5)
p_contact	probability	Hostile-contact likelihood from on-board CV/radar, 0.00-1.00
t	unix seconds	NTP-synced timestamp per observation. e.g. 1700000340
bearing, heading, signal_strength	sensor	Richer telemetry; NOT used by the proof, stays on drone

Physical sensor readings. Only (x, y, p_contact, t) feed the proof. Bearing / heading / signal-strength are operational tradecraft that never leave the drone.

▼ transforms into ▼

Stage 1 Cairo program input (witness) — program_input.json		OFF-CHAIN
Field	Type	Example / Notes
mission_id, drone_id	int	e.g. 2, 2 (bravo). Identify mission + drone-in-swarm (1..N)
strip_x_start / _x_end	int	Assigned vertical strip bounds, x-axis
strip_y_start / _y_end	int	Assigned strip bounds, y-axis
strip_total_cells	int	Total cells in the strip (denominator for coverage)
coverage_min	per mille	950 = require $\geq 95\%$ strip coverage
p_min	basis points	7000 = reject any contact with $p \geq 0.70$
time_window	seconds	360 = mission must finish within 6 min
ts_start, n_cells	int	Mission start time; number of cells swept
cells_x[], cells_y[]	int[]	Parallel arrays of swept-cell coordinates
cells_p_contact[]	int[]	Per-cell contact probability (basis points)
cells_ts[]	int[]	Per-cell timestamps
cells_nonce	felt252 (252-bit)	Random per run — makes the commitment HIDING, not just binding

JSON witness fed to safe_area_verify.cairo. The cells_nonce is the privacy nonce: without it, H would be brute-forceable by an attacker who guessed the cells (Pedersen 1991 hiding).

▼ transforms into ▼

Stage 2 Hiding Pedersen commitment H (computed inside Cairo)		OFF-CHAIN
Field	Type	Example / Notes
H	felt252	0x055d4a0e56c1875e13e8eff57589305bc5bcda38cce164d6bf2343f76c2ea427

H = Pedersen-chain over (cells_x || cells_y || cells_p_contact || cells_ts || cells_nonce). Hiding (nonce) + binding (any cell change alters H). This is the anchor that links the L2 record, the proof, and the L1 fact to one specific cell set without revealing it.

▼ transforms into ▼

Stage 3 Cairo public output — 8 felts (serialize_word order)		OFF-CHAIN
Field	Type	Example / Notes
[0] mission_id	felt252	2
[1] drone_id	felt252	2 (bravo)
[2] strip_x_start	felt252	Strip bound echoed so L1 can check the right strip was swept
[3] strip_x_end	felt252	
[4] strip_y_start	felt252	
[5] strip_y_end	felt252	
[6] verdict_bool	felt252	1 = SAFE (all 4 constraints held); 0 = UNSAFE
[7] commitment_H	felt252	0x055d4a0e...c2ea427

The program ALWAYS produces a valid proof — verdict_bool carries the truth. A failed mission yields a valid proof with verdict_bool = 0, so UNSAFE outcomes are recorded explicitly rather than as 'no proof at all'.

▼ transforms into ▼

Stage 4 L2 transaction — ConvoyProtocol.submit_commitment		L2
Field	Type	Example / Notes
mission_id	felt252	Stark-curve ECDSA signed via the drone's OZ Account contract
drone_id	u8	1..N (drone index in the swarm)
commitment_H	felt252	0x055d4a0e...c2ea427 (from Stage 2)
verdict_bool	u8	0 or 1
proof	felt252[]	The STARK proof payload (or a reference), delegated to the Cairo verifier

The drone authenticates on L2 with Stark-curve ECDSA. The contract verifies the strip bounds match the drone's assigned strip ($\text{strip_width} = \text{zone_w} / \text{n_drones}$) and records the verdict.

▼ transforms into ▼

Stage 5 L2 storage — ConvoyProtocol state		L2
Field	Type	Example / Notes
MissionSpec	struct	zone_x, zone_y, zone_w, zone_h, n_drones, strip_width
StripBounds	struct	x_start, x_end, y_start, y_end (per drone, derived from zone)
verdicts	Map((mid,drone)→u8)	PENDING / SAFE / UNSAFE per drone
commitments	Map((mid,drone)→felt252)	H_i per drone (hiding)

L2 is the convoy's private operational ledger — readable by the convoy via Pathfinder, NOT exposed to L1. When all N drones are SAFE, the contract emits an L1-bound message.

▼ transforms into ▼

Stage 6 Stone STARK proof — proof.json		OFF-CHAIN
Field	Type	Example / Notes
proof body	JSON ~800 KB	Merkle roots + FRI commitments + OODS evals + query answers
layout	string	starknet (Cairo VM layout 6)
n_steps	int	65536 (Cairo VM trace length)
(raw cells)	—	ABSENT: cells were the private witness, never in the proof

cpu_air_prover output. ~802 KB for our run. Passed cpu_air_verifier off-chain (verification: PASSED). Commits to the execution TRACE, not the inputs.

▼ transforms into ▼

Stage 7 EVM-adapted proof — evm_proof.json		OFF-CHAIN
Field	Type	Example / Notes
annotated proof	JSON ~860 KB	stark_evm_adapter gen-annotated-proof output
proof_hex	hex	Serialized proof bytes
public_input	object	memory_segments, public_memory, n_steps

The annotated form the splitter consumes. Re-encodes the Stone proof into EVM-friendly structure without changing what's proved.

▼ transforms into ▼

Stage 8 Split contract args — main_proof_contract_args.json		OFF-CHAIN
Field	Type	Example / Notes
proof_params	uint256[~12]	FRI configuration (n_queries, layer sizes, PoW bits)
proof	uint256[~690]	The STARK proof body as field elements
task_metadata	uint256[~1]	Per-task program-hash metadata
cairo_aux_input	uint256[~30]	Cairo public input (programHash, outputHash, z, alpha, ...)
cairo_verifier_id	uint256	6 (layout 6)

Produced by stark_evm_adapter's split_fri_merkle_statements(). These four arrays are exactly what Verifier.registerSafeProof accepts; cairo_verifier_id is stored immutably on the Verifier.

▼ transforms into ▼

Stage 9	L1 call — Verifier.registerSafeProof(SafeProofInputs, 4 arrays)		L1
Field	Type	Example / Notes	
programHash	bytes32	0xcc28abddd73fddb3b39b9f55da9ac0ad1bffa802592ac694586137ee964c5215	
outputHash	bytes32	0xd8c56e2c4e4826426c446b712170944096b8992cf54ede3f2e63238624454f3c	
missionId	uint256	2	
droneIndex	uint8	2 (1..nDrones)	
stripXStart / stripXEnd	uint32	Strip bounds, re-asserted on L1	
stripYStart / stripYEnd	uint32		
verdictBool	uint8	1 = SAFE	
commitment	bytes32	0x055d4a0e...c2ea427 (H_i)	
nSteps	uint256	65536	
+ proofParams, proof, taskMetadata, cairoAuxInput	uint256[] x4	From Stage 8	

Submitted by the relay ship (Stark-curve on L2, but secp256k1 here on L1). felt252 values widen to uint256; the 252-bit hashes map to bytes32. The relay-whitelist gate checks msg.sender.

▼ transforms into ▼

Stage 10	L1 fact registration (inside GpsStatementVerifier + Verifier)		L1
Field	Type	Example / Notes	
factHash	bytes32	0x6b2b9b235356c61ec2e87e59ae6c175fbd851623edb8c40fb3b692f29852f137	
formula	keccak256	keccak256(abi.encodePacked(programHash, outputHash))	
verifiedFacts[factHash]	bool	true after the real STARK math passes on-chain	

The GpsStatementVerifier validates proofParams/proof/taskMetadata/cairoAuxInput against the layout-6 CpuFrilessVerifier, then registers the fact. Verifier.sol re-asserts the thresholds as defence-in-depth before writing the verdict.

▼ transforms into ▼

Stage 11	L1 verdict — Registry.setVerdict		L1
Field	Type	Example / Notes	
verdict[missionId][droneIndex]	bool/u8	SAFE, written only by the bound Verifier	
MissionDeployed / VerdictSet	event	Indexed by missionId + droneIndex for relay subscription	
MissionSafe	event	Emitted when ALL drones in the swarm are SAFE	

Registry is the verdict ledger. Only the Verifier may call setVerdict, and only after a successful proof. Ship D's commander orchestrator watches for the all-SAFE condition.

▼ transforms into ▼

Stage 12	L1 ADVANCE — CommandLog.advance		L1
Field	Type	Example / Notes	
advance(alphaMissionId, bravoMissionId, speed)	call	Signed by the IMMUTABLE commander key (ship D)	
dual-SAFE check	require	Both lanes' verdicts must be SAFE in Registry	
ConvoyAdvance	event	(blockNumber, alphaMissionId, bravoMissionId, speed, commander)	

The terminal step. msg.sender must equal the immutable commander address. On success, ConvoyAdvance fires and all six ships observe it on the shared L1 chain — the convoy advances.

▼ transforms into ▼

Type transitions across the layers

Concept	Off-chain / Cairo	L2 (StarkNet)	L1 (EVM)
field element	felt252 (252-bit)	felt252	uint256
hash / commitment	felt252 (Pedersen)	felt252	bytes32

small ints (coords)	felt (range-checked)	u8 / u16 / u32 / u64	uint8 / uint32
verdict	felt (0/1)	u8 (PENDING/SAFE/U NSAFE)	uint8 / bool
signature	– (witness)	Stark-curve ECDSA	secp256k1 ECDSA
hash function	Pedersen (commitment)	Pedersen / Poseidon	Keccak-256

The privacy boundary is between L2 and L1. The raw cells live off-chain and on the convoy's private L2; only the proof, the commitment H, the strip bounds, and the verdict cross to L1. An observer of L1 learns the verdict and the strip that was cleared, but never the sweep pattern, the per-cell contact probabilities, or any sensor tradecraft.